

MODEL 980A COMPUTER FORTRAN LIBRARY AND IN-LINE ASSEMBLY LANGUAGE

**MANUAL NO. 961961-9744
REVISED 1 MARCH 1973**

**NATIONAL BIOMEDICAL RESEARCH FOUNDATION
Georgetown University Medical Center
3900 Reservoir Road, N. W.
Washington, D. C. 20007**



**TEXAS INSTRUMENTS
INCORPORATED**

JUN 15 1975

Copyright 1973

By

Texas Instruments Incorporated

All Rights Reserved

PRINTED IN U.S.A.

The information and/or drawings set forth in this document and all rights in and to inventions disclosed herein and patents which might be granted thereon disclosing or employing the materials, methods, techniques or apparatus described herein are the exclusive property of Texas Instruments Incorporated.

No disclosure of the information or drawings shall be made to any other person or organization without the prior consent of Texas Instruments Incorporated.

ENGINEERING CHANGE NOTICE

DATE	ECN	LEVEL	DESCRIPTION
2-7-73	373977	E	Add note to top of page 4-6.

TABLE OF CONTENTS

Section		Page
I	INTRODUCTION	
1-1	Scope	1-1
1-2	Related Material	1-1
II	FORTRAN LIBRARY	
2-1	Scope	2-1
2-2	Floating-Point Arithmetic Package	2-1
2-2.1	Purpose	2-1
2-2.2	Floating-Point Numbers	2-2
2-2.3	Usage	2-3
2-3	External and Intrinsic Functions	2-5
2-3.1	General	2-5
2-3.2	Argument Setup	2-6
2-3.3	Basic External Functions	2-8
2-3.4	Intrinsic Functions	2-8
2-4	Use of FORTRAN Run-Time Subroutines	2-9
2-4.1	Power Routines	2-9
2-4.2	Mode Conversion Routines	2-10
2-4.3	Real Arithmetic	2-11
2-4.4	Double-Precision Arithmetic	2-12
2-4.5	Complex Arithmetic	2-12
2-5	Input/Output	2-12
2-6	Functional Subroutine Descriptions	2-14
2-6.1	Real Functional Subroutines	2-14
2-6.1.1	Square Root	2-14
2-6.1.2	Sine/Cosine	2-15
2-6.1.3	Arctangent	2-16
2-6.1.4	Natural Logarithm	2-16
2-6.1.5	Exponential	2-17
2-6.1.6	Hyperbolic Tangent	2-18
2-6.1.7	Arctangent of Two Arguments	2-18
2-6.1.8	Base Ten Logarithm	2-18
2-6.2	Double-Precision Functional Subroutines	2-19
2-6.2.1	Double-Precision Square Root	2-19
2-6.2.2	Sine/Cosine	2-19
2-6.2.3	Arctangent	2-21
2-6.2.4	Natural Logarithm	2-22
2-6.2.5	Exponential	2-22

Section

Page

2-6.2.6	Arctangent of Two Arguments	2-23
2-6.2.7	Base Ten Logarithm	2-23
2-6.3	Complex Functional Routines	2-23
2-6.3.1	Complex Square Root	2-24
2-6.3.2	Complex Sine/Cosine	2-24
2-6.3.3	Complex Logarithm	2-24
2-6.3.4	Complex Exponential	2-25
2-6.4	Fortran Intrinsic Functions	2-25
2-6.4.1	Absolute Value Routines	2-25
2-6.4.2	Truncation Routines	2-26
2-6.4.3	Remaindering	2-27
2-6.4.4	Maximum/Minimum Routines	2-27
2-6.4.5	Transfer of Sign Routines	2-31
2-6.4.6	Positive Difference Routines	2-32
2-6.4.7	Mode Conversion Routines	2-32
2-6.5	Run-Time Subroutine Algorithms	2-34
2-6.5.1	Power Routines	2-34
2-6.5.2	Mode Conversion Routines	2-36
2-6.5.3	Special Procedures	2-39
III	ASSEMBLY LANGUAGE LIBRARY REFERENCES	
3-1	Scope	3-1
3-2	Calls to External and Intrinsic Functions	3-1
3-3	Calls to Run-Time Subroutines	3-2
3-4	Calls to Floating-Point Package	3-3
3-5	Calls to FFEDIT	3-4
3-6	Assembly Language Subroutines for FORTRAN Linkage	3-9
IV	IN-LINE ASSEMBLY LANGUAGE STATEMENTS	
4-1	Scope	4-1
4-2	Register-To-Memory Instructions	4-1
4-3	Register-To-Register Instructions	4-3
4-4	Indicator Skip Instructions	4-4
4-5	Register Skip Instructions	4-4
4-6	Shift Instructions	4-4
4-7	Pseudo-Op Instructions	4-5
4-8	Restrictions	4-6

Section	Page
APPENDIX A	
TABLE A-1. FORTRAN BASIC EXTERNAL FUNCTIONS	
TABLE A-2. FORTRAN INTRINSIC FUNCTIONS	
TABLE A-3. FORTRAN FORMAT EDITOR	
APPENDIX B	
RUN-TIME ERROR MESSAGES	

LIST OF TABLES

Table No.		Page
1-1	Related Manuals	1-1
2-1	Legal FORTRAN I/O Operations	2-13

SECTION I

INTRODUCTION

1-1 SCOPE.

This is the third in a series of three manuals describing the FORTRAN compiler for the Model 980A Computer, and it presupposes a knowledge of 980A assembly language. This manual describes the FORTRAN library programs available and tells how to use them at both the FORTRAN and assembly language level. The library includes the intrinsic and basic external functions of ANSI standard FORTRAN, as well as the floating-point arithmetic package and the FORTRAN format editor. All routines are in a relocatable format and may be linked to assembly language programs by using the link editor. Routines using the standard calling sequence described in Paragraph 2-3.1 are also available to FORTRAN programmers. The program descriptions include the name and function of each entry point, the calling sequence, and any called subroutines.

This manual also lists the In-line Assembly Language (IAL) statements that can be interspersed with the 980A FORTRAN statements. In addition, the coding conventions that must be observed for an assembly language program to be FORTRAN callable are given.

1-2 RELATED MATERIAL.

The first two manuals in this series describe those features of 980A FORTRAN that are common to most FORTRAN compilers and may be skimmed by the experienced FORTRAN programmer.

The information needed to compile and execute a FORTRAN program may be found in the Basic System Use and Operation Manual. A list of all 980A manuals available and a glossary of terms may be found in the Model 980A Computer Software Manual Index. The manual numbers to be used when ordering related material are given in Table 1-1.

TABLE 1-1. RELATED MANUALS

	MANUAL NO.
Model 980A Computer FORTRAN Arithmetic and Control	961961-9740
Model 980A Computer FORTRAN Declarations and Input/Output	961961-9742
Model 980A Computer Basic System Use and Operation	961961-9710
Model 980A Computer Software Manual Index	961961-9792

SECTION II

FORTRAN LIBRARY

2-1 SCOPE.

The FORTRAN library contains a collection of commonly used function subprograms which have been prepared for use by the FORTRAN programmer. These subprograms are used as function references in arithmetic statements, rather than in CALL statements. These pre-defined functions are divided into two groups called *basic external functions* and *intrinsic functions*. In addition to the function subprograms supplied, there is a group of programs required at run-time to assist in the execution of a FORTRAN program. These programs are not explicitly referenced by a FORTRAN program, but are called by the compiler as required to perform some subsidiary function, such as exponentiation or mode conversion.

Many library routines operate on data contained in the *floating-point accumulator*, *complex accumulator*, or hardware register A. They also leave results in these registers. The floating-point accumulator and complex accumulator are actually memory locations within the floating-point arithmetic package. They are symbolically identified by the names ACC\$ and EXP\$ for the mantissa and exponent of real and double-precision numbers, and CACC\$ and CACC2\$ for the real and imaginary portions of complex numbers. The location MODE\$ is used to set the mode of the floating-point arithmetic. All of these symbols are externally defined and may be referenced by an assembly language program.

2-2 FLOATING-POINT ARITHMETIC PACKAGE.

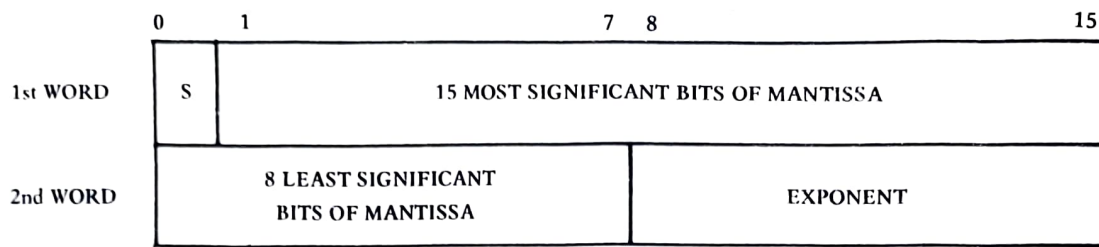
2-2.1 PURPOSE. The floating-point package provides a group of floating-point arithmetic subroutines which are entered by trapped operation codes. These codes are available to the assembly language programmer through the use of OPD (Operation Define) assembler directives. The package operates in either the single-precision or double-precision mode according to the contents of location MODE\$ which is a defined location that can be referenced externally.

MODE	OPERATION MODE
0	Single-Precision; Program Counter Relative Usage
1	Single-Precision; Base Register Relative (FORTRAN mode)
2	Double-Precision; Program Counter Relative
3	Single-Precision; Base Register Relative
4	Double-Precision; Base Register Relative

NOTE

Consult Appendix B for run-time error messages printed by the floating-point arithmetic package.

2-2.2 FLOATING-POINT NUMBERS. Single-precision quantities require two consecutive 16-bit computer words.

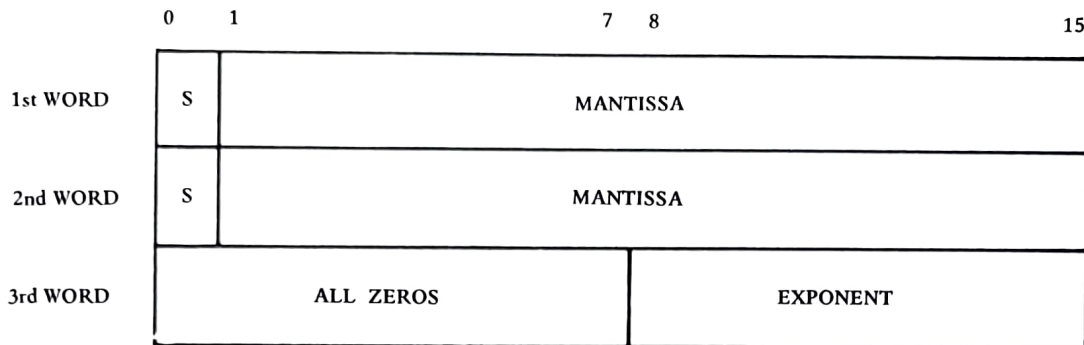


Bit zero of the first word is the algebraic sign. Bits 1-15 are the first fifteen bits of the mantissa in two's complement form. Bits 0-7 of the second word are the next eight bits of the mantissa. Bits 8-15 of the second word represent the power of two exponent biased at X'0080'. The binary point is assumed to be between the sign bit and first mantissa bit. The following are examples of single-precision floating-point quantities (in hexadecimal).

QUANTITY	FLOATING-POINT REPRESENTATION	
0.5	4000	0080
1.0	4000	0081
0.25	4000	007F
-0.5	C000	0080
0.0	0000	0000

Note that zero is represented by a double word of zeros. In addition, all floating-point quantities are assumed to be normalized; i.e., the high order bit of the mantissa is significant. The pair 8000 00 is not allowed. If it results from a floating-point operation, the mantissa is shifted right one place and the exponent increased by one.

Double-precision floating-point quantities require three consecutive words.



Bit zero of the first word is the algebraic sign of the mantissa. Bits 1-15 of this word are the most significant 15 bits of the mantissa. Bit zero of the second word is the same as bit zero of the first word. Bits 1-15 of the second word are the low order 15 bits of the mantissa. Bits 0-7 of the third word are zero. Bits 8-15 of the third word are the power of two exponent, biased at X'0080'. Double-precision zero consists of zeros in all three words.

All operations are in the single-precision or double-precision mode according to the contents of MODE\$. Single-precision mantissas provide up to 23 bits of accuracy (approximately seven decimal digits) and double-precision mantissas provide up to 30 bits of accuracy (approximately nine decimal digits).

2-2.3 USAGE. The floating-point package is entered by trapped operation codes. These codes cause the instruction in memory location two to be executed. When not running under Basic System control, the user must set locations two and three to:

```
SSB
DATA    FP980A
```

If running under Basic System control, the user must initialize the illegal instruction trap by a service call

```
@LDM    = FP980A
DATA    >C382
```

A FORTRAN-generated program causes the trap to be set without any action on the part of the user.

Addressing in modes 0 and 2 is always program counter relative. The user may specify indirect addressing and/or indexing as usual. If an instruction is indexed, the index register content is doubled or tripled according to whether MODE\$ is set for single-precision or double-precision operation. In other words the index register is set for indexing to the array item as opposed to memory location. For example, if the index register contains a 3 and indexing is indicated, the index value actually used is 6 for single-precision mode or 9 for double-precision mode. Modes 3 and 4 are identical except they use base register relative addressing.

In mode 1 (used by FORTRAN-generated programs) all addressing is base relative. Three modes of addressing are:

$B + D$ = Base register + displacement

$(B + D)$ = Base register + displacement, indirect

$(B + D + X)$ = Base register + displacement + index register, indirect.

Indexing is by memory location, not by array item as in modes 0 and 2.

NOTE

In all modes no registers are preserved except the index and base registers.

Floating-point operations normally involve an operand from memory and an operand in the pseudo floating accumulator in FP980A. This accumulator consists of three memory locations. The first two of these contain the mantissa (single-precision or double-precision) of a floating-point quantity. The first of these two locations is labeled ACC\$. The third location contains the power of two exponent biased at X'0080', and is labeled EXP\$. Both ACC\$ and EXP\$ are entry points of FP980A.

Any operation which results in a biased exponent outside the range (0 to FF) causes the hardware overflow indicator to be set upon exit from the package. If the biased exponent is negative (underflow), it is set to zero. If the biased exponent is greater than FF, it is set to FF. If a divide operation is indicated and the divisor is zero, the overflow indicator is turned on and no operation is executed.

Floating-point instructions can be assembled as normal machine instructions by using OPD assembler directives. The following operations are provided:

FLD OPD >D000 – Floating Load. Load ACC\$-EXP\$ with the contents of the effective address.

FLN OPD >D100 – Floating Load Negative. Load ACC\$-EXP\$ with the negative of the contents of the effective address.

FST OPD >E000 – Floating Store. Replace the contents of the effective address with the contents of ACC\$-EXP\$.

FCP OPD >E100 – Floating Compare. Compare the contents of the effective address to ACC\$-EXP\$ and set the hardware compare indicators to reflect the result.

FAD OPD >E800 – Floating Add. Add the contents of the effective address to the contents of ACC\$-EXP\$ and place the sum in ACC\$-EXP\$.

FSB OPD >E900 – Floating Subtract. Subtract the contents of the effective address from the contents of ACC\$-EXP\$ and place the difference in ACC\$-EXP\$.

FMP OPD >F000 – Floating Multiply. Multiply the contents of ACC\$-EXP\$ by the contents of the effective address and place the product in ACC\$-EXP\$.

FDV OPD >F100 – Floating Divide. Divide the contents of ACC\$-EXP\$ by the contents of the effective address and place the quotient in ACC\$-EXP\$.

NEG OPD >C580 – Negate. Place the negative of ACC\$ in ACC\$. EXP\$ is not affected.

2-3 EXTERNAL AND INTRINSIC FUNCTIONS.

2-3.1 GENERAL. This portion of the library consists of all subroutines necessary to support ANSI Standard FORTRAN in addition to special routines for fetching arguments and performing double-precision arithmetic.

Associated with these subroutines is a set of hardware and software registers for receiving arguments and returning function values.

REGISTER	TYPE
Hardware Register A	Integer
Software ACC\$-EXP\$	Real and Double-Precision
Software CACC\$, CACC2\$	Complex – Real Part in CACC\$, Imaginary Part in CACC2\$

Routines which are called explicitly in FORTRAN use the sequence:

```
BRL    *$+1
DATA   Subroutine Name
DATA   Number of Arguments
DATA   Argument Address 1
DATA   Argument Address 2
      .
      .
      .
DATA   Argument Address n
```

This calling sequence is the standard sequence used for any subroutine called or referenced by a FORTRAN program.

Subroutines which are called implicitly by FORTRAN (power routines for example) do not require an argument count. The calling sequence is

```
BRL    *$+1
DATA   Subroutine Name
DATA   Address of Argument 1
DATA   Address of Argument 2
      .
      .
      .
DATA   Address of Argument n
```

Subroutines of this type are described in Subsection 2-4.

2-3.2 ARGUMENT SETUP. Argument addresses for both calling sequences are fetched by a single subroutine named ARGUS\$ (argument setup). Its functions are:

- a. Save the current floating-point mode in MODE\$ on a pushdown list.
- b. Establish a new mode for the calling routine.
- c. Save the return on a pushdown list.
- d. Restore MODE\$ and the return address at time of exit.

ARGSU\$ has five entry points; one pair for each type of calling sequence, and one common to both calling sequences.

- a. SET1\$ is the initial entry point if the calling sequence contains an argument count. Upon entry, the A register should contain the return address to the calling program and the E register should contain the value to be placed in MODE\$. If the value of MODE\$ is immaterial, the value in the E register can be any value. Upon return from ARGSU\$, the A register contains the address of the next argument and the M register contains the argument count.
- b. SET2\$ is the initial entry point if the calling sequence does not contain an argument count.
- c. ARG\$ — Subsequent entries to ARGSU\$ are made at ARG\$ for the purpose of obtaining the next argument address which will be returned in the A register.

- d. XIT1\$ – When a subroutine which initially calls ARG\$ at SET1\$ is finished, it should exit to XIT1\$. This causes the value of MODE\$ to be restored and returns control to the main program.
- e. XIT2\$ – Serves the same purpose as XIT1\$ for subroutines which initially call ARG\$ at SET2\$.

To illustrate the use of ARG\$, consider the construction of a subroutine which forms the sum of a number of floating-point numbers. Let the entry point of the subroutine be SUM. The calling sequence is:

```

BRL    *$+1
DATA   SUM
DATA   N          ARGUMENT COUNT
DATA   Address of First Argument
.
.
.
DATA   Address of Last Argument

```

The subroutine would appear

SUM	RMO	L,A	LINK REGISTER TO A
	LDE	=0	SET MODE\$ TO SINGLE-PRECISION
	BRL	*SET1	GO TO ARG\$ AT SET1\$
	STA	*ARGADR	ADDRESS OF FIRST ARGUMENT
	RCO	M,X	NEGATIVE OF ARG COUNT TO X
	FLD	*ARGADR	LOAD FIRST ARGUMENT
	BRU	BIX	
LOOP	BRL	*ARG	GET NEXT ARGUMENT ADDRESS FROM
	STA	ARGADR	ARG\$ - ENTRY POINT ARG\$
	FAD	*ARGADR	ADD NEXT ARGUMENT
BIX	BIX	LOOP	BUMP ARGUMENT COUNT
	BRU	*XIT1	ALL ARGUMENTS USED - EXIT TO
ARGADR	BSS	1	ARG\$ AT XIT1\$
XIT1	DATA	XIT1\$	
ARG	DATA	ARG\$	
SET1	DATA	SET1\$	
	REF	ARG\$, SET1\$, XIT1\$	

If the calling sequence does not contain an argument count, different entry points to ARG\$ are used. Suppose the larger of two double-precision quantities must be determined, and it must be left in the floating accumulator. The calling sequence is:

BRL	*\$+1	
DATA	MAX	
DATA	ARG1	ADDRESS OF FIRST ARGUMENT
DATA	ARG2	ADDRESS OF SECOND ARGUMENT

The subroutine MAX would appear as follows:

MAX	RMO	L,A	LINK TO A
	LDE	=2	SET MODE\$ FOR DOUBLE-PRECISION
	BRL	*SET	
	STA	ARGADR	ADDRESS OF FIRST ARGUMENT
	FLD	*ARGADR	FIRST ARGUMENT TO FLOATING ACC.
	BRL	*ARG	FETCH AND STORE ADDRESS
	STA	ARGADR	OF SECOND ARGUMENT
	FCP	*ARGADR	COMPARE ARGUMENTS
	SLT		
	BRU	*XIT	FIRST ARGUMENT GREATER - EXIT
	FLD	*ARGADR	SECOND ARGUMENT GREATER - PLACE IN FLOATING ACC.
	BRU	*XIT	
SET	DATA	SET2\$	
ARGADR	BSS	1	
XIT	DATA	XIT2\$	
ARG	DATA	ARG\$	
	REF	SET2\$, XIT2\$, ARG\$	

2-3.3 BASIC EXTERNAL FUNCTIONS. Appendix Table A-1 summarizes the basic external functions available in the FORTRAN library. These subroutines are available to both the FORTRAN and assembly language programmer. Section 3-2 specifically covers their use in assembly language programs. The subprograms included in this group are characterized by the fact that they implement certain commonly recognized mathematical functions.

2-3.4 INTRINSIC FUNCTIONS. Appendix Table A-2 summarizes the intrinsic functions available in the FORTRAN library. These subroutines are also available to both the FORTRAN and assembly language programmer. Section 3-2 specifically covers their use in assembly language programs. The subprograms included in this group are characterized by the fact that they implement certain useful logical procedures.

2-4 USE OF FORTRAN RUN-TIME SUBROUTINES.

In addition to the function subprograms supplied, there is a group of programs required at run-time to assist in the execution of a FORTRAN program. These programs are not explicitly referenced by a FORTRAN program, but are called by the compiler as required to perform some subsidiary function.

The run-time routines include the power and mode conversion subroutines, the real, double-precision, and complex arithmetic subroutines, and the input/output subroutines. These may all be referenced by assembly language programs, provided the proper calling sequence is observed.

At the completion of any arithmetic operation on real, double-precision, or complex data, the results are located in the *floating-point accumulator* or *complex accumulator* — memory locations within the floating-point arithmetic package as previously described in Subsection 2-2. Likewise, input to arithmetic routines is often assumed to be in one or the other of these pseudo-registers. They are symbolically identified by the names ACC\$ and EXP\$ for the mantissa and exponent of real and double-precision numbers and CACC\$ and CACC2\$ for the real and imaginary portions of complex numbers. The location MODE\$ is used to set the mode of the floating-point arithmetic.

2-4.1 POWER ROUTINES. A series of power routines is available for computing

BASE**EXP

The mode of the result is the same as the mode of the base. The result is returned in the appropriate hardware or software register as below:

BASE	RESULT IN
Integer	A Register
Real	ACC\$, EXP\$
Double-precision	ACC\$, EXP\$
Complex	CACC\$, CACC2\$

The calling sequence is:

BRL	*\$+1
DATA	<i>Subroutine Name</i>
DATA	BASE
DATA	EXP

The following routines are available:

NAME	BASE	EXPONENT
\$F4IIP	Integer	Integer
\$F4RIP	Real	Integer
\$F4DIP	Double	Integer
\$F4CIP	Complex	Integer
\$F4IRP	Integer	Real
\$F4RRP	Real	Real
\$F4DRP	Double	Real
\$F4IDP	Integer	Double
\$F4RDP	Real	Double
\$F4DDP	Double	Double

2-4.2 MODE CONVERSION ROUTINES. Mode conversion routines are available for conversions between various arithmetic modes. These routines assume that the argument is in the hardware or software register appropriate to the mode of the argument. The result is returned in the hardware/software register appropriate to the mode of the function. The registers and associated modes are as follows:

MODE	REGISTER
Integer	A Register
Real	ACC\$, EXP\$
Double	ACC\$, EXP\$
Complex	CACC\$, CACC2\$

Since arguments are assumed to be in the proper registers, the calling sequence consists only of a call to the appropriate routine. For example:

```
BRL    *$+1
DATA   $F4IRC
```

The routines available are listed below:

NAME	FUNCTION
\$F4IRC	Integer-to-real conversion
\$F4IDC	Integer-to-double conversion
\$F4ICC	Integer-to-complex conversion
\$F4RIC	Real-to-integer conversion
\$F4RDC	Real-to-double conversion
\$F4RCC	Real-to-complex conversion
\$F4DIC	Double-to-integer conversion
\$F4DRC	Double-to-real conversion
\$F4DCC	Double-to-complex conversion
\$F4CIC	Complex-to-integer conversion
\$F4CRC	Complex-to-real conversion
\$F4CDC	Complex-to-double conversion

2-4.3 REAL ARITHMETIC. No special set of run-time subroutines is required to perform arithmetic on real numbers, rather floating-point arithmetic is performed by the use of the illegal instruction trap. The mode of operation is determined by the value in the location denoted symbolically as MODE\$.

MODE\$	OPERATION MODE
0	Real – Program counter relative
1	Real – Base register relative (word indexing)
2	Double-precision – Program counter relative
3	Real – Base register relative (item indexing)
4	Double-precision – base register relative

MODE\$ is initially set to 1; therefore, the user must reset MODE\$ for the required mode if mode 1 is not to be used.

The addressing mode is determined by the I and X fields as in the example below for MODE\$ = 1.

IX	ADDRESSING
00	B+D
10	(B+D)
01	B+D+X
11	Illegal

Note that base register addressing is not specified through use of the B bit. It can be selected only by use of the mode word MODE\$.

In mode 1 indexing is by word (normal indexing). In modes 0 and 2 indexing is by item. Thus, if the X register contains 2, the value 4 is added to B+D in mode 0 and the value 6 is added in mode 2. In any of the above modes, the following operation codes are used. They may be defined by an OPD by the assembly language user.

FLD	OPD	>D000	FLOATING LOAD
FLN	OPD	>D100	FLOATING LOAD NEGATIVE
FST	OPD	>E000	FLOATING STORE
FAD	OPD	>E800	FLOATING ADD
FSB	OPD	>E900	FLOATING SUBTRACT
FMP	OPD	>F000	FLOATING MULTIPLY
FDV	OPD	>F100	FLOATING DIVIDE
FNG	OPD	>C580	FLOATING NEGATE
FCP	OPD	>E100	FLOATING COMPARE

The codes thus defined are used in the same fashion as normal operation codes.

2-4.4 DOUBLE-PRECISION ARITHMETIC. Double-precision arithmetic can also be performed by subroutines, in addition to the illegal instruction traps described in 2-4.3. The calling sequence is:

```
BRL    *$+1
DATA   Subroutine Name
DATA   Argument
```

The routines available are listed below.

NAME	FUNCTION
\$F4DST	Double-precision store
\$F4DAD	Double-precision add
\$F4DLD	Double-precision load
\$F4DSB	Double-precision subtract
\$F4DMP	Double-precision multiply
\$F4DDV	Double-precision divide
\$F4DCP	Double-precision compare
\$F4DCS	Double-precision change sign of accumulator

The double-precision package requires FP980A and ARGSU\$ and is named DARIT\$.

2-4.5 COMPLEX ARITHMETIC. Complex arithmetic operations are performed by the set of routines listed below. The results of complex operations are stored in locations denoted symbolically by CACC\$ and CACC2\$. The calling sequence is:

```
BRL    *$+1
DATA   Subroutine Name
DATA   Argument
```

NAME	FUNCTION
\$F4CLD	Complex load
\$F4CST	Complex store
\$F4CAD	Complex add
\$F4CSB	Complex subtract
\$F4CMP	Complex multiply
\$F4CDV	Complex divide
\$F4CCS	Complex change sign (negate CACC\$, CACC2\$)

The complex arithmetic package requires FP980A and ARGSU\$, and it is named CARIT\$.

2-5 INPUT/OUTPUT.

Model 980A FORTRAN input/output functions are handled by a set of 16 routines, grouped into one program, FFEDIT (FORTRAN format editor). Table 2-1 defines the legal operations which may be performed using FFEDIT. The format editor programs are as follows:

ENTRY	FUNCTION
\$F4EFI	End file
\$F4RWD	Rewind
\$F4SIO	Stop input/output
\$F4RBN	Read – Binary
\$F4RBD	Read – ASCII
\$F4WBN	Write – Binary
\$F4WBD	Write – ASCII
\$F4IIO	Integer/logical input/output
\$F4RIO	Real input/output
\$F4CIO	Complex input/output
\$F4DIO	Double-precision input/output
\$F4IUA	Integer/logical unsubscripted array input/output
\$F4RUA	Real unsubscripted array input/output
\$F4CUA	Complex unsubscripted array input/output
\$F4DUA	Double-precision unsubscripted array input/output
\$F4BSP	Backspace

Two other required FORTRAN functions, stop and pause, are included in the library subroutine nam \$F4STP . Their entry names and functions are shown below:

ENTRY	FUNCTION
\$F4STP	Stop
\$F4PAU	Pause

Appendix Table A-3 contains a summary of allowable I/O operations.

TABLE 2-1 LEGAL FORTRAN I/O OPERATIONS

Device	Read Binary	Write Binary	Read ASCII	Write ASCII	Rewind	Backspace	End of File
Teletypewriter or Data Terminal			X	X			
Paper Tape Reader	X		X				
Paper Tape Punch		X		X			X
Card Reader	X		X				
Card Punch		X		X			X
Line Printer				X			
Magnetic Tape	X	X	X	X	X	X	X
Disc	X	X	X	X	X	X	X

2-6 FUNCTIONAL SUBROUTINE DESCRIPTIONS.

This section describes the algorithms used to implement the FORTRAN basic external functions (the "mathematical" functions), the FORTRAN intrinsic functions, and the run-time subroutines.

2-6.1 REAL FUNCTIONAL SUBROUTINES. The calling sequence for all real functional subroutines is:

```
BRL    *$+1
DATA   Subroutine Entry
DATA   1
DATA   Argument Address
```

The value of the function is left in ACC\$-EXP\$. All functional subroutines require the following programs:

NAME	FUNCTION
FP980A	Floating-point package
ARGSU\$	Argument setup

2-6.1.1 SQUARE ROOT.

```
Name  SQR$
Entry  SQRT
```

If the argument is zero, there is an immediate exit from the subroutine. If the argument is negative, it is complemented prior to computation of the root.

If the floating-point number A is represented by $N \cdot 2^p$, its square root is $N^{1/2} \cdot 2^{p/2}$. If the exponent of A is odd, N is shifted right one place and p is increased by one so p is even. A first approximation to the square root of N , x_0 , is computed by

$$x_0 = \frac{1}{2}(1+A)$$

The square root of N is determined by iterating on Newton's method four times. If x_n is an approximation to the square root of A , then a better approximation, x_{n+1} is given by

$$x_{n+1} = \frac{1}{2}(x_n + A/x_n)$$

2-6.1.2 SINE/COSINE.

Name SINCOS

Entry SIN, COS

The argument, y , is divided by $\pi/2$ to yield

$$y/(\pi/2) = I + F$$

where I is the integer part of the quotient and F is the fractional part. The result is an integer, Q , in the range (0,3) defining the quadrant in which the argument lies. Sin y , or cos y , as desired, equals sin z where z is given by the following tables.

I is reduced to $Q \pmod{4}$ — only the low order two bits are kept.

X POSITIVE

Q	Sine	Cosine
0	$z=F$	$z=(1-F)$
1	$z=1-F$	$z=-F$
2	$z=-F$	$z=-(1-F)$
3	$z=-(1-F)$	$z=F$

X NEGATIVE

Q	Sine	Cosine
0	$z=F$	$z=1+F$
1	$z=-(1+F)$	$z=F$
2	$z=-F$	$z=-(1+F)$
3	$z=1+F$	$z=-F$

Sine z is computed from

$$\sin z = \sum_{n=0}^4 a_{2n+1} z^{2n+1}$$

where

$$a_1 = 1.5707963$$

$$a_3 = -0.6459637$$

$$a_5 = 0.07968968$$

$$a_7 = -0.004673766$$

$$a_9 = 0.00015148482$$

2-6.1.3 ARCTANGENT.

Name ATAN\$

Entry ATAN

The arctangent is computed from the polynomial approximation

$$\arctan x = \sum_{n=0}^7 a_{2n+1} x^{2n+1} \quad (-1 \leq x \leq 1)$$

where

$$a_1 = 0.99999\ 93329$$

$$a_3 = -0.33329\ 85605$$

$$a_5 = 0.19946\ 53599$$

$$a_7 = -0.13908\ 53351$$

$$a_9 = 0.09642\ 00441$$

$$a_{11} = -0.05590\ 98861$$

$$a_{13} = 0.02186\ 12288$$

$$a_{15} = -0.00405\ 40580$$

If the argument is greater than 1, the argument $z=1/x$ is used in the polynomial evaluation and

$$\arctan x = \pm \pi/2 + \arctan z$$

The sign of $\pi/2$ is chosen to be opposite that of $\arctan z$.

2-6.1.4 NATURAL LOGARITHM.

Name LOG\$

Entry ALOG

The natural logarithm is computed from the relations

$$\log_2 x \cdot 2^p = \log_2 x + p$$

$$\ln x = \ln 2 \cdot \log_2 x$$

where $\log_2 x$ is computed by

$$\log_2 x = -\frac{1}{2} + a_1 z + a_3 z^3 + a_5 z^5 \quad (\frac{1}{2} \leq x \leq 1)$$

$$z = \frac{\sqrt{2x-1}}{\sqrt{2x+1}}$$

where

$$a_1 = 2.88539129$$

$$a_3 = 0.96147063$$

$$a_5 = 0.59897865$$

2-6.1.5 EXPONENTIAL.

Name EXP\$

Entry EXP

The exponential function computation depends on the relations

$$e^x = e^{(x \log_2 e) \log_e 2} = e^{(I+F) \log_e 2}$$

where

I = integer part of $x \log_2 e$

F = fractional part of $x \log_2 e$

continuing

$$e^{(I+F) \log_e 2} = e^{I \log_e 2} e^{F \log_e 2} = 2^I e^{F \log_e 2}$$

where

$$0 \leq F \log_e 2 < \log_e 2$$

so that

$$I \leq e^{F \log_e 2} < 2.$$

Letting $F \log_e 2 = y$

$$e^{-y} = \sum_{n=0}^7 a_n y^n$$

where

$a_0 = 1.00000\ 00000$
 $a_1 = -0.99999\ 99995$
 $a_2 = 0.49999\ 99206$
 $a_3 = -0.16666\ 53019$
 $a_4 = 0.04165\ 73475$
 $a_5 = -0.00830\ 13598$
 $a_6 = 0.00132\ 98820$
 $a_7 = -0.00014\ 13161$

Then $e^x = 2^{I/e^{-y}}$

2-6.1.6 HYPERBOLIC TANGENT.

Name `TANH$`
Entry `TANH`

The hyperbolic function is computed using `EXP$` from the relation

$$\tanh x = (e^{2x} - 1) / (e^{2x} + 1)$$

2-6.1.7 ARCTANGENT OF TWO ARGUMENTS.

Name `ATAN2$`
Entry `ATAN2`

The arctangent of the ratio of the two arguments is computed using `ATAN$`. The result is then modified to produce an angle in the range $(-\pi, \pi)$ according to the algebraic signs of the two arguments as shown below.

Arg 1	Arg 2	Result
+	+	No modification
+	-	Add π
-	+	No modification
-	-	Subtract π

2-6.1.8 BASE TEN LOGARITHM.

Name `ALOG10`
Entry `ALOG10`

The natural logarithm is computed using `ALOG$` and multiplied by $\log_{10} e$.

2-6.2 DOUBLE-PRECISION FUNCTIONAL SUBROUTINES. All double-precision functional routines use double-precision floating-point arithmetic for computation.

2-6.2.1 DOUBLE-PRECISION SQUARE ROOT.

Name DSQRT\$

Entry DSQRT

The square root routine is based on Newton's method. If the argument is an odd number, the mantissa is shifted right one place and the exponent increased by one. If it is an even number, no alteration is made. Therefore, the floating-point representation is of the form

$$x = f \cdot 2^{2b}$$

where $0.25 \leq x < 1$. Then

$$\sqrt{x} = \sqrt{f} \cdot 2^b$$

An initial approximation of $\sqrt{f}$ is determined by

$$f_0 = af + b$$

where

$$a = 0.875, b = 0.27863 \text{ for } 0.25 \leq x < 0.5$$

$$\text{or } a = 0.578125, b = 0.421875 \text{ for } 0.5 \leq x < 1$$

This initial value is refined by two Newton iterations

$$f_1 = \frac{1}{2}(f_0 + f/f_0)$$

$$f_2 = \frac{1}{2}(f_1 + f/f_1)$$

2-6.2.2 SINE/COSINE.

Name DSICO\$

Entry DSIN, DCOS

Let the argument x be divided by 2π so that

$$x/2\pi = I + F$$

where I is the integer portion of the quotient and F the fractional part. Then

$$x = 2\pi I + 2\pi F$$

and $\sin x = \sin(2\pi I + 2\pi F) = \sin 2\pi F$

and $\cos x = \cos(2\pi I + 2\pi F) = \cos 2\pi F$

The cosine is computed from

$$\cos 2\pi F = g(z)$$

where $z = \frac{1}{4} - F$ for $0 \leq F \leq \frac{1}{2}$

$$z = F - \frac{3}{4} \text{ for } \frac{1}{2} \leq F < 1$$

and $\sin 2\pi F = g(z)$

where $z = F$ for $0 \leq F < \frac{1}{4}$

$$z = \frac{1}{2} - F \text{ for } \frac{1}{4} \leq F < \frac{3}{4}$$

$$z = F - 1 \text{ for } \frac{3}{4} \leq F < 1$$

$g(z)$ is the polynomial in z given by

$$g(z) = \sum_{n=0}^5 a_{2n+1} z^{2n+1}$$

where

$$a_1 = 6.283185307$$

$$a_3 = -41.34170212$$

$$a_5 = 81.60522621$$

$$a_7 = -76.70428132$$

$$a_9 = 42.00980573$$

$$a_{11} = -14.39413537$$

2-6.2.3 ARCTANGENT.

Name DATAN\$

Entry DATAN

The routine is based upon an approximation for $\arctan z$ where $-0.23 \leq z \leq 0.23$. Range reduction is accomplished by the relations

$$\arctan x = a_k = \arctan [(x - b_k)/(x b_k + 1)], x \geq 0$$

where

$$a_k = (k\pi/7)b_k = \tan a_k$$

and k is chosen so

$$\tan [(2k-1)\pi/14] < x < \tan [(2k+1)\pi/14], k=1, 2, 3$$

After determining the value of k which satisfies the above inequality, the transformation

$$z = [(x - b_k)/(x b_k + 1)]$$

places z in the range $-\tan \pi/14 \leq z \leq \tan \pi/14$.

Arctan z is then computed from

$$\arctan z = \sum_{n=0}^4 a_{2n+1} z^{2n+1}$$

where

$$a_1 = 1.00000\ 0000$$

$$a_3 = 0.33333\ 3271$$

$$a_5 = 0.19999\ 0568$$

$$a_7 = 0.14235\ 1775$$

$$a_9 = 0.09992\ 3312$$

and $\arctan x = a_k + \arctan z$.

If $x < 0$ the relation $\arctan (-x) = -\arctan x$ allows evaluation of a positive argument and subsequent negation to obtain the final result.

2-6.2.4 NATURAL LOGARITHM.

Name DLOG\$

Entry DLOG

The floating-point argument is of the form

$$x = 2^k F$$

where $\frac{1}{2} \leq F < 1$. This can be rewritten as $x = 2^j g$, where $\sqrt{2}/2 \leq g < \sqrt{2}$, by setting $j = k-1$ and $g = 2F$, if $F < \sqrt{2}/2$; otherwise, $j = k$ and $g = F$.

Then $\ln x = j \ln 2 + \ln g$.

To compute $\ln g$ let

$$z = (g-1)/(g+1)$$

then $\ln x = k \ln 2 + z g(z)$

$$\text{where } g(z) = \sum_{n=0}^4 b_n z^{2n}$$

and

$$b_0 = 2.0$$

$$b_2 = 0.6666666564$$

$$b_4 = 0.400018841$$

$$b_6 = 0.284535727$$

$$b_8 = 0.125$$

2-6.2.5 EXPONENTIAL.

Name DEXP\$

Entry DEXP

From the identity $e^x = 2^{x \log_2 e}$ and letting $x \log_2 e = n + d + z$ we get $e^x = 2^n 2^d 2^z$ where

n is the integer part of $x \log_2 e$

d is the discrete fractional part 0/8 to 7/8

z is the fractional part $-1/8 < z < 1/8$

then 2^z is approximated by $f(z)$ where

$$f(z) = \sum_{n=0}^5 a_n z^n$$

and

$a_0 = 1.0$
 $a_1 = 0.69314\ 7181$
 $a_2 = 0.24022\ 6486$
 $a_3 = 0.05550\ 41054$
 $a_4 = 0.00962\ 173987$
 $a_5 = 0.00133\ 37729375$

therefore $e^x = 2^n 2^d f(z)$

2-6.2.6 ARCTANGENT OF TWO ARGUMENTS.

Name DATN2\$
 Entry DATAN2

The arctangent of the ratio of the two arguments is computed using DATN2\$. The result is modified according to the signs of the arguments to produce an angle in the range $(-\pi, \pi)$ as shown below.

Arg 1	Arg 2	Result
+	+	No modification
+	-	Add π
-	+	No modification
-	-	Sub π

2-6.2.7 BASE TEN LOGARITHM.

Name DLN10\$
 Entry DLG10

The natural logarithm is computed using DLOG\$ and the result is multiplied by $\log_{10} e$.

2-6.3 COMPLEX FUNCTIONAL ROUTINES.

The calling sequence to all complex functional routines is

```

BRL    *$+1
DATA   Subroutine Entry
DATA   1
DATA   Argument Address
  
```

All leave the result in CACC\$, CACC2\$.

2-6.3.1 COMPLEX SQUARE ROOT.

Name CSQRT\$

Entry CSQRT

Let the argument be represented in rectangular form $x+iy$. Then

$$(x + iy)^{1/2} = (x^2 + y^2)^{1/2} e^{i\theta/2} = R e^{i\theta/2} = R(\cos \theta/2 + i \sin \theta/2)$$

where

$$\theta/2 = \frac{1}{2} \arctan (y/x)$$

The required function is computed by evaluating $R(\cos \theta/2 + i \sin \theta/2)$ and the result is left in the complex accumulator. CSQRT\$ requires ARGSU\$, ATAN\$, ALOG\$, SIN COS, and EXP\$, in addition to CARIT\$.

2-6.3.2 COMPLEX SINE/COSINE.

Name CSICO\$

Entry CSIN, CCOS

The sine and cosine of complex quantities are determined from the identities.

$$\begin{aligned} \sin(x+iy) &= \sin x \cos iy + \cos x \sin iy \\ &= \sin x \cosh y + i \cos x \sinh y \end{aligned}$$

$$\begin{aligned} \cos(x+iy) &= \cos x \cos iy - \sin x \sin iy \\ &= \cos x \cosh y - i \sin x \sinh y \end{aligned}$$

$$\sinh y = (e^y - e^{-y})/2$$

$$\cosh y = (e^y + e^{-y})/2$$

2-6.3.3 COMPLEX LOGARITHM.

Name CLOG

Entry CLOG

The complex logarithm is computed from the relation

$$\ln(x+iy) = \frac{1}{2} \ln(x^2 + y^2) + i \arctan y/x$$

CLOG requires ARGSU\$, ATAN, and ALOG.

2-6.3.4 COMPLEX EXPONENTIAL.

Name CEXP

Entry CEXP

The complex exponential is computed from the relation

$$e^{x+iy} = e^x (\cos y + i \sin y)$$

It requires EXP\$ and SINCOS as well as ARG\$U\$.

2-6.4 FORTRAN INTRINSIC FUNCTIONS. This section describes the FORTRAN intrinsic functions. The standard calling sequence is used for all intrinsic functions.

```
BRL    *$+1
DATA   Subroutine Name
DATA   Number of Arguments
DATA   Address of first argument
      .
      .
DATA   Address of last argument
```

All of these subroutines use ARG\$U\$ to set the floating-point mode and get argument addresses. The result is returned in the appropriate hardware or software register according to the function type.

2-6.4.1 ABSOLUTE VALUE ROUTINES.

- a. Integer Absolute.

Name IABS\$

Entry IABS

The argument is placed in the A register and complemented if negative.

- b. Real Absolute.

Name ABS\$

Entry ABS

The argument is placed in ACC\$-EXP\$. If the mantissa is negative, it is complemented.

- c. Double-Precision Absolute.

Name DABS\$

Entry DABS

The floating mode is set to double-precision and ABS\$ is entered at ABS\$A with an unconditional branch.

- d. Complex Modulus.

Name CABS\$

Entry CABS

The complex quantity $z = x+iy$ is converted to a real quantity by the relation

$$|z| = (x^2 + y^2)^{1/2}$$

and stored in ACC\$-EXP\$. SQRT\$ is required.

2-6.4.2 TRUNCATION ROUTINES.

- a. Real Truncation

Name AINT\$

Entry AINT

The integer portion of the argument is obtained by using \$F4RIC. This integer is then converted to a real quantity using \$F4IRC and the result left in ACC\$-EXP\$.

- b. Integer Truncation.

Name INT\$

Entry INT

The integer portion of the argument is obtained by using \$F4RIC. The result is left in the A register.

- c. Double-Precision/Integer Truncation.

Name IDINT\$

Entry IDINT

The integer portion of the double-precision argument is obtained by calling \$F4DIC, and the result is left in the A register.

2-6.4.3 REMAINDERING. The remaindering or modulus routines are functions of two arguments, a_1 and a_2 . The function determined by these routines is

AMOD(a_1, a_2)

DMOD(a_1, a_2) = $a_1 - |a_1/a_2| a_2$

MOD(a_1, a_2)

Where $|x|$ means the magnitude of the integer x does not exceed the magnitude of x and the sign is the same as x .

a. Integer Remaindering.

Name MOD\$

Entry MOD

The function described above is computed with integer arguments and returns an integer result.

b. Real Remaindering.

Name AMOD\$

Entry AMOD

The ratio a_1/a_2 of the real arguments is determined and the integer portion extracted using \$F4RIC and \$F4IRC. The modulus is computed as above.

c. Double-Precision Remaindering.

Name DMOD\$

Entry DMOD

The ratio a_1/a_2 of the double-precision arguments is computed and the integer portion extracted using \$F4IDC. This integer is converted back to double-precision by \$F4IDC and the modulus computed as above.

2-6.4.4 MAXIMUM/MINIMUM ROUTINES. The following routines select the maximum or minimum value of an arbitrary number of quantities presented by an argument list in the calling sequence. The routines differ in the mode of the argument and the mode of the function. If only one argument is specified, it will be returned by the called subroutine.

The routines are divided into two classes: those with integer arguments and those with real or double-precision arguments. Each class has one routine with two entry points. Other routines in the same class load registers with initialization information and call the general routine of the same class at the second entry point.

- a. Maximum - Integer Argument - Real Function.

Name AMAX0\$

Entry AMAX0, AMAX0\$

When entered at AMAX0, the following initial conditions are set:

A register = SLE instruction for max selection

M register = 1 to return a real value

Control then goes to the entry point AMAX0\$. The maximum integer included in the argument list is selected and converted to a real quantity by \$F4IRC and returned in ACC\$-EXP\$. This routine is used by all max/min routines with integer arguments.

- b. Maximum - Integer Argument - Integer Function.

Name MAX0\$

Entry MAX0

The following initial conditions are set:

A register = SLE instruction for max selection.

M register = 0 to return an integer value.

Control is then transferred to AMAX0\$ at AMAX0\$. The maximum integer value included in the argument list is selected and returned in the A register.

- c. Minimum - Integer Argument - Real Function.

Name AMIN0\$

Entry AMIN0

The following conditions are set:

A register = SGE instruction for min selection.

M register = 1 to return a real value.

Control then goes to AMAX0\$ at AMAX0\$. The minimum integer included in the argument list is selected and converted to a real quantity by \$F4IRC and returned in ACC\$-EXP\$.

d. Minimum - Integer Argument - Integer Function.

Name MIN0\$

Entry MIN0

The following initial conditions are set:

A register = SGE instruction for minimum selection.

M register = 0 to return integer value.

Control then goes to AMAX0\$ at AMAX0\$. The minimum integer included in the argument list is selected and returned in the A register.

e. Maximum - Real Argument - Real Function.

Name AMAX1\$

Entry AMAX1,AMAX\$

When entered at AMAX1, the following initial conditions are set.

A register = SLE instruction for max selection.

M register = 0 to return a real value.

E register = 0 for single-precision mode.

Control then goes to the entry point AMAX\$. The maximum real quantity included in the argument list is selected and returned in ACC\$-EXP\$. This routine is used by all max/min routines which have real or double-precision arguments.

f. Maximum - Real Argument - Integer Function.

Name MAX1\$

Entry MAX1

When entered, the following initial conditions are set.

A register = SLE instruction for max selection.

M register = 1 to return an integer value.

E register = 0 for single-precision mode.

Control is then transferred to AMAX1\$ at AMAX\$. The maximum integer included in the argument list is selected and converted to an integer value by \$F4RIC. This integer is returned in the A register.

- g. Maximum - Double-Precision Argument - Double-Precision Function.

Name DMAX1\$

Entry DMAX1

When entered the following initial conditions are set.

A register = SLE instruction for max selection.

M register = 0 to return double-precision value.

E register = 2 for double-precision mode.

Control is then transferred to AMAX1\$ at AMAX\$. The maximum double-precision quantity included in the argument list is selected and returned in ACC\$-EXP\$.

- h. Minimum - Real Argument - Real Function.

Name AMIN1\$

Entry AMIN1

The following initial conditions are set.

A register = SGE instruction for min selection.

M register = 0 to return real value.

E register = 0 for single-precision mode.

Control is then returned to AMAX1\$ at AMAX\$. The minimum real quantity included in the argument list is selected and returned in ACC\$-EXP\$.

- i. Minimum - Real Argument - Integer Function.

Name MIN1\$

Entry MIN1

The following initial conditions are set.

A register = SGE instruction for min selection.

M register = 1 to return integer value.

Control is then transferred to AMAXI\$ at AMAX\$. The minimum real quantity included in the argument list is selected and converted to an integer by \$F4RIC. The integer is returned in A.

j. Minimum - Double-Precision Argument - Double-Precision Function.

Name DMIN1\$

Entry DMIN1

The following initial conditions are set.

A register = SGE instruction for min selection.

M register = 0 to return double-precision value.

E register = 2 for double-precision mode.

Control is then transferred to AMAXI\$ at AMAX\$. The minimum double-precision quantity included in the argument list is selected and returned in ACC\$-EXP\$.

2-6.4.5 TRANSFER OF SIGN ROUTINES. The following three subroutines require two arguments, a_1 and a_2 . Their function is to transfer the sign of argument a_2 to the argument a_1 .

a. Sign - Integer Argument - Integer Function.

Name ISIGN\$

Entry ISIGN

The sign of a_2 is transferred to a_1 as described above and the integer result is returned in the A register.

b. Sign - Real Argument - Real Function.

Name SIGN\$

Entry SIGN

The sign of a_2 is transferred to a_1 as described above and the real result is returned in ACC\$-EXP\$.

c. Sign - Double-Precision Argument - Double-Precision Function.

Name DSIGN\$

Entry DSIGN

The sign of a_2 is transferred to a_1 as described above and the double-precision result is returned in ACC\$-EXP\$.

2-6.4.6 POSITIVE DIFFERENCE ROUTINES. The following two routines determine the positive difference of two arguments a_1 and a_2 as

$$f(a_1, a_2) = a_1 - \text{minimum}(a_1, a_2)$$

- a. Positive Difference - Integer Argument - Integer Function.

Name IDIM\$

Entry IDIM

The positive difference of a_1, a_2 is constructed as described above and the integer result is returned in the A register.

- b. Positive Difference - Real Argument - Real Function.

Name DIM\$

Entry DIM

The positive difference of a_1, a_2 is constructed as described above and the real result is returned in ACC\$-EXP\$.

2-6.4.7 MODE CONVERSION ROUTINES. The following routines are provided, for conversion between various modes.

- a. Integer to Real - Integer Argument - Real Function.

Name FLOAT\$

Entry FLOAT

The integer presented by the argument list is converted to a real quantity by \$F4IRC and returned in ACC\$-EXP\$.

- b. Real to Integer - Real Argument - Integer Function.

Name FIX\$

Entry IFIX

The real quantity presented by the calling sequence is converted to an integer by \$F4RIC and the result is returned in the A register.

- c. Double to Real - Double-Precision Argument - Real Function.

Name SNGL\$

Entry SNGL

The double-precision quantity whose address appears in the calling sequence is placed in ACC\$-EXP\$.

- d. Real Part of Complex - Complex Argument - Real Function.

Name REAL\$

Entry REAL

The real part of the complex quantity specified in the calling sequence is placed in ACC\$-EXP\$.

- e. Imaginary Part of Complex - Complex Argument - Real Function.

Name AIMAG\$

Entry AIMAG

The imaginary part of the complex quantity specified by the calling sequence is placed in ACC\$-EXP\$.

- f. Real to Double Precision - Real Argument - Double Precision Function.

Name DBLE\$

Entry DBLE

The real quantity specified by the calling sequence is placed in ACC\$-EXP\$.

- g. Real to Complex - Two Real Arguments - Complex Function.

Name CMPLX\$

Entry CMPLX

The first of the two real arguments is placed in CACC\$ as the real part of a complex quantity and the second real argument is placed in CACC2\$ as the imaginary part of a complex quantity.

- h. Complex Conjugate - Complex Argument - Complex Function.

Name CONJ\$
Entry CONJG

The complex quantity specified in the calling sequence is placed in the complex accumulator - CACC\$, CACC2\$ with its imaginary part negated.

2-6.5 RUN-TIME SUBROUTINE ALGORITHMS. This section describes the computational procedure for the run-time subroutines.

2-6.5.1 POWER ROUTINES. All power routines have the same calling sequence.

BRL *\$+1
DATA *Subroutine Name*
DATA *Base*
DATA *Exponent*

The result is left in the hardware or software register corresponding to the mode of the base. Integer base routines return the result in the A register while real and double-precision base routines return the result in ACC\$-EXP\$ and complex base routines return the result in CACC\$,CACC2\$. All arguments are fetched by ARGSU\$.

- a. Integer Base to Integer Power.

Name \$F4IIP
Entry \$F4IIP

If the exponent is greater than zero and the base is nonzero, the power is computed by multiplication. Otherwise the following table shows the results of \$F4IIP.

BASE	POWER	RESULT
0	Any	0
1	<0	1
-1	<0	$(-1)^P$
≠0	0	1
≠0,≠1	<0	0

The result is left in the A register.

b. Integer Base to Real Power.

Name \$F4IRP

Entry \$F4IRP

The real exponent is converted to integer by \$F4RIC and the result and the original base are used as arguments for \$F4IIP. The result is left in the A register.

c. Integer Base to Double Power.

Name \$F4IDP

Entry \$F4IDP

The double-precision exponent is converted to an integer using \$F4DIC and the result and the original base are used as arguments for \$F4IIP. The result is left in the A register.

d. Real Base to Integer Power.

Name \$F4RIP

Entry \$F4RIP

The power is computed by repeated multiplication. If the base is zero, a result of zero is returned. If the base is nonzero and the exponent is negative, the reciprocal of the base is used in the computation. The result is returned in ACC\$-EXP\$.

e. Real Base to Real Power.

Name \$F4RRP

Entry \$F4RRP

If the base is zero, a result of zero is returned. If the base is negative, it is made positive prior to computation. The logarithm of the base is calculated using ALOG\$ and it is multiplied by the exponent. This is exponentiated using EXP\$. The result is returned in ACC\$-EXP\$.

f. Real Base to Double Power.

Name \$F4RDP

Entry \$F4RDP

The double-precision exponent is placed in ACC\$-EXP\$ and a floating store in the single-precision mode is used to store it as a real quantity. The computational procedure is the same as for \$F4RRP. The result is left in ACC\$-EXP\$.

- g. Double-Precision Base to Integer Power.

Name \$F4DIP

Entry \$F4DIP

The computational procedure is the same as that for \$F4RIP except that double-precision arithmetic is used. The result is returned in ACC\$-EXP\$.

- h. Double-Precision Base to Real Power.

Name \$F4DRP

Entry \$F4DRP

The real exponent is fetched and stored in double-precision format. If the base is zero, a result of zero is returned and if it is negative, it is made positive. The power is computed using DLOG\$ and DEXP\$ to evaluate $D^R = e^{R \ln D}$. The result is returned in ACC\$-EXP\$.

- i. Double-Precision Base to Double-Precision Power.

Name \$F4DDP

Entry \$F4DDP

If the base is zero, a result of zero is returned. If it is negative, it is made positive. The power is computed using DLOG\$ and DEXP\$ to evaluate $D_1^{D_2} = e^{D_2 \ln D_1}$.

- j. Complex Base to Integer Power.

Name \$F4CIP

Entry \$F4CIP

The base is fetched using the complex load and store subroutines (\$F4CLD, \$F4CST). If the base is zero, a result of zero is returned. If the base is nonzero and the exponent is zero, the real and complex parts of the result are set to one and zero, respectively. If the exponent is negative, the reciprocal of the base is taken. The power is then computed by repeated complex multiplication using \$F4CMP. The result is returned in CACC\$-CACC2\$.

2-6.5.2 MODE CONVERSION ROUTINES. A number of routines are provided for conversion between modes; integer, real, double-precision, and complex. The calling sequence consists only of

BRL *\$+1
DATA *Subroutine Name*

These routines assume that the argument is in the appropriate hardware or software register and return the result in the hardware or software register appropriate to the output mode as shown below.

MODE	REGISTER
Integer	Hardware Register A
Real	Software Floating Accumulator ACC\$-EXP\$
Double-Precision	Software Floating Accumulator ACC\$-EXP\$
Complex	Software Complex Accumulator CACC\$, CACC2\$

a. Integer to Real Conversion.

Name \$F4IRC
Entry \$F4IRC

If the integer is zero, ACC\$-EXP\$ are set to zero. Otherwise, the integer is normalized and stored double-precision in ACC\$. The shift count is subtracted from X'8F' and the result placed in EXP\$.

b. Integer to Double Conversion.

Name \$F4IDC
Entry \$F4IDC

This routine consists of an unconditional branch to \$F4IRC.

c. Integer to Complex Conversion.

Name \$F4ICC
Entry \$F4ICC

The integer is converted to real using \$F4IRC and the result placed in CACC\$, CACC2\$ is set to zero.

d. Real to Integer Conversion.

Name \$F4RIC
Entry \$F4RIC

If the real number is less than one, a zero result is returned. Otherwise, the fractional part of the mantissa is shifted off. If the argument is greater than $2^{15} - 1$, only the high order 15 bits are returned.

e. Real to Double-Precision Conversion.

Name \$F4RDC

Entry \$F4RDC

The argument is assumed to be in ACC\$-EXP\$. In the software floating accumulator, both real and double-precision quantities have the same format so no action is required.

f. Real to Complex Conversion.

Name \$F4RCC

Entry \$F4RCC

The real argument is placed in CACC\$, CACC2\$ is set to zero.

g. Double to Integer Conversion.

Name \$F4DIC

Entry \$F4DIC

This routine calls \$F4RIC which does the conversion.

h. Double to Real Conversion.

Name \$F4DRC

Entry \$F4DRC

Since the formats of real and double-precision in ACC\$-EXP\$ are the same, no action is taken.

i. Double to Complex Conversion.

Name \$F4DCC

Entry \$F4DCC

The double-precision argument is placed in CACC\$ as a real quantity and CACC2\$ is set to zero.

j. Complex to Integer Conversion.

Name \$F4CIC

Entry \$F4CIC

The real part of the argument (CACC\$) is placed in ACC\$-EXP\$. \$F4RIC is then called to effect the conversion to integer.

- k. Complex to Real Conversion.

Name \$F4CRC

Entry \$F4CRC

The real part of the argument is placed in ACC\$-EXP\$.

- l. Complex to Double Conversion.

Name \$F4CDC

Entry \$F4CDC

The routine consists of an unconditional branch to \$F4CRC.

2-6.5.3 SPECIAL PROCEDURES. The run-time subroutine portion includes six special procedures which, in general, cannot be classified in any of the previously described groups. Their functions and names are as follows.

- a. Assigned Go To. A call to \$F4AGT is generated by FORTRAN whenever an assigned GO TO statement is encountered. The calling sequence is

```
BRL    *$+1
DATA   $F4AGT
DATA   Address of Transfer Location
```

\$F4AGT transfers control to the transfer location.

- b. Computed Go To. A call \$F4CGT is generated by FORTRAN whenever a computed GO TO statement is encountered. Control is transferred based on the variable value and label list; i.e., control is transferred as directed by that label in the list the ordering of which corresponds to the value of the variable. The link instruction is followed by 2+N arguments, where N is the number of labels in the list.

The calling sequence is

```
BRL    *$+1
DATA   $F4CGT
DATA   Address of Variable
DATA   Number of Labels
DATA   First Label Address
      .
      .
DATA   Nth Label Address
```

- c. Assign. A call to \$F4ASG is generated by FORTRAN whenever an ASSIGN statement is encountered. It associates the indicated label with the indicated variable. For example

Assign 5 to K

would generate

```
BRL    *$+1
DATA   $F4ASG
DATA   Address of K
DATA   Address of Label 5
```

- d. Global Dummy Setup. A call to \$F4GSU is generated at the start of all FORTRAN generated subprograms. Its major function is the setup of actual argument addresses to replace the meaningless values in the dummy argument referenced cells. Dummy argument referenced cells are allocated in groups of three. Two of these cells contain meaningless information and one contains either zero (scalar variables) or an offset value (array variables).

\$F4GSU stores actual argument addresses in one of the temporary cells and subtracts the offset from this value and places this result in the second temporary cell.

The calling sequence is

```
BRL    *$+1
DATA   $F4GSU
DATA   N
DATA   Return Temp.
DATA   DMY
```

DMY is the first cell of the sequence of three word blocks described above. The statement DATA N indicates N arguments and RETURN TEMPORARY is for holding the return address. \$F4GSU stores $(X)+X+1$ into RETURN TEMPORARY where $X=(E)+1$ = address of argument count in the calling sequence.

\$F4GSU is also called at the start of a main program for the purpose of setting the base register and initializing the floating-point trap.

- e. Dummy Array Setup. \$F4ASU is executed for each adjustably dimensioned array in a subprogram. An attempt is made to do this by \$F4GSU but the results are meaningless. Only the base address of the dummy triplet contains valid information. \$F4ASU must calculate the array offset and the base minus offset.

In addition, subscript calculation coding generated by the compiler references values not yet established in temporary cells. \$F4ASU, therefore, computes subscript multipliers and the array size and stores them in temporary cells provided for that purpose.

- f. Local Dummy Setup. \$F4LSU is executed at the start of all statement functions. Its major function is to set up actual argument addresses. The calling sequence is

```
BRL    *$+1
DATA   $F4LSU
DATA   Number of Local Dummies
DATA   Address of First Temp.
```


SECTION III

ASSEMBLY LANGUAGE LIBRARY REFERENCES

3-1 SCOPE.

This section describes the calling sequences required to use a library function or procedure in an assembly language program. Most of the library functions described in Section II may be linked with assembly language programs. The exceptions are those library subroutines written exclusively to implement procedures which have meaning only in the FORTRAN context. The six exceptions are as follows:

NAME	PURPOSE
\$F4GSU	Dummy variables are set up in a subprogram.
\$F4LSU	Dummy variables are set up in an arithmetic function statement.
\$F4ASU	Dummy dimensions are set up for an array.
\$F4ASG	The ASSIGN statement is performed.
\$F4AGT	The assigned GO TO statement is performed.
\$F4CGT	The computed GO TO statement is performed.

3-2 CALLS TO EXTERNAL AND INTRINSIC FUNCTIONS.

In order to make use of the several functions described as external and intrinsic, the assembly language program must contain the same calling sequence as that generated by the FORTRAN compiler. The required sequence is as follows:

```
BRL    *$+1
DATA   NAME
DATA   N
DATA   ARG1
DATA   ARG2
      .
      .
      .
DATA   ARGN
```

where NAME is the name of the function being referenced, N is the number of arguments in the call, and ARG1, ARG2, etc., are the labels of the cells containing the actual arguments for the function. For example, suppose it is necessary to find the integer value of the result of floating-point operations on real numbers. Further, suppose this result is stored in the two cells starting in the location labeled VALUE. The proper calling sequence is:


```

BRL    *$+1
DATA   IFIX
DATA   1
DATA   VALUE

```

The integer value of the variable VALUE is returned in the hardware accumulator, register A.

Similarly, to convert two real numbers into one complex number, the calling sequence is:

```

BRL    *$+1
DATA   CMPLX
DATA   2
DATA   REAL1
DATA   REAL2

```

where REAL1 and REAL2 are the labels for the locations of the two real numbers. The result of CMPLX is returned in the complex software accumulator CACC\$ and CACC2\$.

In the two previous examples, it is assumed that the assembly language program contains a statement defining IFIX and CMPLX as external references, i.e.,

```

REF    IFIX,CMPLX

```

3-3 CALLS TO RUN-TIME SUBROUTINES.

The FORTRAN compiler is furnished a list of the run-time subroutines, together with the fixed number of arguments required for each. It is, therefore, possible for the compiler to generate a shortened version of the standard calling sequence that does not include the argument count. In order for the calling sequence to be properly handled by the routine being called, the assembly language programmer must match the calling sequence generated by the compiler. The routines affected are the power, mode conversion, double-precision, and complex arithmetic routines.

Execution of the following calling sequence causes the double-precision number at DPV to be raised to the real power at ALPHA and leaves the real value result in the real software accumulator ACCS and EXPS.

```

BRL    *$+1
DATA   $F4DRP
DATA   DPV
DATA   ALPHA

```

Refer to Paragraph 2-4.1 for a complete list of power routines.

In order to accomplish mode conversion, the number to be converted must be in the proper register prior to execution of the call, and the result is returned in the appropriate register. No arguments are required. The coding

```
LDA    INT
BRL    *$+1
DATA   $F4IRC
```

causes the integer in the hardware accumulator A to be converted to the internal floating-point format, and the result to be left in the software accumulator ACC\$ and EXP\$ when control is returned to the calling program. Refer to Paragraph 2-4.2 for a complete list of mode conversion routines.

Two complex numbers may be added by executing the following code:

```
BRL    *$+1
DATA   $F4CLD
DATA   CX1
BRL    *$+1
DATA   $F4CAD
DATA   CX2
BRL    *$+1
DATA   $F4CST
DATA   CRESUL
```

The complex number at CX1 is added to the complex number at CX2 and the sum is placed in CRESUL.

Refer to Paragraph 2-4.5 for a complete list of complex arithmetic routines.

3-4 CALLS TO FLOATING-POINT PACKAGE.

Use of the floating-point operations does not involve a calling sequence. The floating-point routines are entered by using trapping instructions that cause control to be routed to floating-point subroutines. The assembly language pseudo-op OPD is used by the programmer to define the trapping instructions. The arithmetic mode must be set, and the entry address of the floating-point package (FP980A) must be set with the proper supervisor call. The 980A Basic System provides interrupt trap decoding and floating-point subroutine linkage. A program to evaluate the equation

$$Y = 2 * X ** 2 + 7 * X + 6$$

might appear as follows:

```
FLD    OPD    >D000
FMP    OPD    >F000
FAD    OPD    >E800
FST    OPD    >E000
      REF    MODE$,FP980A
X      BSS    2
Y      BSS    2
SIX    DATA  >6000,>0083
TWO    DATA  >4000,>0082
SEVEN  DATA  >7000,>0083
START  LDA    =0
      @STA   MODE$
      @LDM   =FP980A
      DATA  >C382
      FLD    TWO
      FMP    X
      FAD    SEVEN
      FMP    X
      FAD    SIX
      FST    Y
      .
      .
      .
```

3-5 CALLS TO FFEDIT.

The FORTRAN format editor is available as a relocatable library subroutine for use in assembly language programs. Formats must be provided by the user in the form of data statements, i.e.,

```
FMT1    DATA  '(2HA=F10.5)'
```

The portion enclosed between single quotes must be in accordance with the usual syntax of FORTRAN format statements.

There are 16 entry points to the FORTRAN format editor as shown in Section 2-5.

NOTE

Consult Appendix B for run-time error messages printed by the format editor.

Calling sequences for the non-I/O functions are shown below:

BRL	*\$+1	END FILE
DATA	\$F4EFI	
DATA	UNIT NO.	
BRL	*\$+1	REWIND
DATA	\$F4RWD	
DATA	UNIT NO.	
BRL	*\$+1	BACKSPACE
DATA	\$F4BSP	
DATA	UNIT NO.	

All input/output operations are initiated by a call to one of four subroutines, \$F4WBN, \$F4WBD, \$F4RBN, or \$F4RBD. The calling sequence is illustrated below.

BRL	*\$+1
DATA	\$F4WBD
DATA	FMT1
DATA	UNIT NO.

For each element of the argument list, there must be a call to an input/output routine. The last call in an I/O operation must be to the stop I/O (\$F4SIO) routine.

To illustrate:

```

      BRL    *$+1      INITIATE OUTPUT
      DATA  $F4WBD    OPERATION ON LOGICAL UNIT
      DATA  FMT21     3 USING FMT21
      DATA  THREE
      BRL    *$+1      OUTPUT I
      DATA  $F4IIO
      DATA  I
      BRL    *$+1      OUTPUT A
      DATA  $F4RIO
      DATA  A
      BRL    *$+1      STOP I/O
      DATA  $F4SIO
      .
      .
      .
FMT21  DATA  '(I4,F10.5) '
THREE  DATA  3

```

The following example illustrates the input of 12 integer values from logical unit 5, into the 3 x 4 integer array I.

```

      BRL    *$+1
      DATA  $F4RBD
      DATA  FORMAT
      DATA  FIVE
      BRL    *$+1
      DATA  $F4IUA
      DATA  12
      DATA  I
      BRL    *$+1
      DATA  $F4SIO
      .
      .
      .
FORMAT DATA  '(I4) '
FIVE    DATA  5
I        BSS   12

```

As a further example, the following code is used to output a string of Hollerith characters to logical unit 6 .

```

REF    $F4WBD,$F4SIO
BRL    *$+1
DATA   $F4WBD
DATA   FMTA      ADDRESS OF FORMAT STATEMENT
DATA   UNITN     ADDRESS OF UNIT NO.
BRL    *$+1
DATA   $F4SIO
.
.
.
FMTA   DATA    '(16HHOLLERITH OUTPUT)'
UNITN  DATA    6

```

Execution of this sequence would cause the line

HOLLERITH OUTPUT

to be written on the specified output unit. Note that the string of information in the DATA statement following FMTA and enclosed in single quotes is identical to the information which would be required in an ordinary FORTRAN format statement. The preceding code is that which would be generated by the FORTRAN compiler from scanning the list associated with the corresponding write statement. In the example, the output is a string of Hollerith characters. No input/output list is involved.

Consider the output of the single integer quantity I in a six-character field using the following code:

```

BRL    *$+1
DATA   $F4WBD      INITIATE ASCII OUTPUT AND
DATA   FMTB        SPECIFY FORMAT AND UNIT NO.
DATA   Address of Unit Number
BRL    *$+1
DATA   $F4IIIO     OUTPUT INTEGER QUANTITY I
DATA   I
BRL    *$+1        TERMINATE OUTPUT
DATA   $F4SIO
.
.
.
I      BSS        1
FMTB   DATA      '(I6)'    FORMAT SPECIFICATION

```

Note that a call to the editor is required for each quantity to be output. If three real quantities A, B, and C are to be read, the following code might be used.

```

BRL    *$+1
DATA   $F4RBD
DATA   FMT5
DATA   Address of Unit Number
BRL    *$+1
DATA   $F4RIO
DATA   A
BRL    *$+1
DATA   $F4RIO
DATA   B
BRL    *$+1
DATA   $F4RIO
DATA   C
BRL    *$+1
DATA   $F4SIO
      .
      .
      .
A      BSS      2
B      BSS      2
C      BSS      2
FMT5   DATA    '(3F10.5)'
```

Logical data are handled by calls to \$F4IIO. On input, the first nonblank character in the field must be a T or F and the value of the variable is set to true or false accordingly. The remainder of the field is ignored. On output, W-1 blanks followed by T or F are output where W is the width of the field from the format specification.

Alphanumeric data is transmitted by calls to \$F4IIO. The number of characters transmitted is limited by the maximum number of characters which can be stored in the space allotted for the variable (2). If the field width exceeds two, leading characters are lost on input and replaced with blanks on output. If the field width is less than two, a blank is inserted after the given character.

The calling sequences for the stop and pause functions are shown below:

	BRL	*\$+1	STOP
	DATA	\$F4STP	
	DATA	N	CONTROL DOES NOT RETURN,
N	DATA	<i>Octal Number</i>	N CAN FOLLOW CALL
	BRL	*\$+1	PAUSE
	DATA	\$F4PAU	
	DATA	N	CONTROL RETURNS, N CANNOT FOLLOW CALL
	.		
	.		
	.		
N	DATA	<i>Octal Number</i>	

where N is the label attached to the DATA statement defining the associated argument. Note that the argument of the stop and pause routines must be an octal number. PAUSE causes the message PAUSE and the specified *octal number* to be printed, then control is returned to the calling program.

3-6 ASSEMBLY LANGUAGE SUBROUTINES FOR FORTRAN LINKAGE.

This section covers the conventions which must be followed when coding an assembly language function or subroutine to be referenced or called by a FORTRAN program. A skeleton example clarifies the Model 980A FORTRAN requirements.

When a call or reference to an assembly language subprogram is found by the FORTRAN processor, the standard calling sequence is generated. As a result, when control is passed to an assembly language subprogram, linkage, return, and argument information can be obtained by noting that the hardware link register contains the address of the argument count, and that the addresses of the arguments follow the argument count.

NOTE

The arguments in the calling sequence are *addresses*, not *values*, of the variables that appear in the CALL statement or function reference.

The FORTRAN compiler assumes that all subroutines preserve the base register. All other registers may be used without saving the contents.

The following skeleton example illustrates a general method to meet the specified conventions, assuming an argument count of three.

```

RETURN  BSS    1
BASE    BSS    1
LOCALB  DATA  RETURN
VAR1    BSS    2
VAR2    BSS    2
VAR3    BSS    2
        BRS    RETURN
START   RMO    L,A
        STA    ARGADR
        LDA    *ARGADR    GET ARGUMENT COUNT
        RCO    A,X
        RAD    X,X        ASSUME REAL VARIABLES
        ADD    =1
        RAD    L,A        CALCULATE AND SAVE
        STA    RETURN    RETURN ADDRESS
        RMO    B,A
        STA    BASE      SAVE BASE REGISTER VALUE
        LDA    LOCALB    SET BASE REGISTER TO
        RMO    A,B        LOCAL VALUE
LOOP    IMO    ARGADR    STEP POINTER TO AN ARGUMENT ADDRESS
        LDA    *ARGADR
        STA    TEMP
        DLD    *TEMP      GET ARGUMENT,
        DST    VAR1+6,X   ASSUMED REAL
        BIX    $+1
        BIX    LOOP
        .
        .
        .

```

{ Continue with processing as required, to implement the desired function.
 If this is a function subprogram, the result is left in the appropriate
 software or hardware register. Finally, the return is made. }

```

        .
        .
        .
        LDE    BASE      RESTORE BASE REGISTER
        RMO    E,B
        BRU    *RETURN
ARGADR  BSS    1
        BSS    1
        END

```

SECTION IV

IN-LINE ASSEMBLY LANGUAGE STATEMENTS

4-1 SCOPE.

In addition to the standard FORTRAN statements defined in the preceding sections, the 980A FORTRAN has been extended to accept a rather large subset of the 980A assembly language mnemonics. This subset is referred to as In-Line Assembly Language (IAL). When the mnemonics are used in a FORTRAN program, the first character of the line (or column 1 of the card) must be the letter A. A four-digit, unsigned number, in characters 2 through 5, is allowed as a statement number. Characters in position 6 are ignored. If a statement is followed by a semicolon, all characters appearing after the semicolon are treated as comments. Comments may be inserted also by using the letter C as the first character of the line.

In IAL statements, the symbols > or \ preceding a digital string identify that string as a hexadecimal number.

4-2 REGISTER-TO-MEMORY INSTRUCTIONS.

The following instructions of the register-to-memory type are allowed. They may be preceded by @ which indicates extended format. The mnemonic codes have the meaning described in the Assembly Language Machine Instructions manual.

ADD	DIV	LDE
AND	DLD	LDM
BIX	DMT	LDX
BRL	DSB	MPY
BRU	DST	STA
CPA	IMO	STE
CPL	IOR	STX
DAD	LDA	SUB

These instructions have two operands. The use of these operands is exactly as with ordinary assembly language programming. The first operand of a register-to-memory instruction may be a scalar or array variable name, plus or minus a decimal integer constant less than or equal to 32,767 or a null field. The first operand may also be a decimal or hexadecimal integer constant. If the instruction is not extended format, this may be a decimal constant in the range -128 to +127, inclusive, or a hexadecimal constant from -80 to +7F, inclusive. If the extended format is used, the number may be in the range -32,768 to +32,767 decimal or 8000 to 7FFF hexadecimal. Labels appearing on FORTRAN statements may also serve as the first operand. This is indicated by a preceding ampersand (&) and may have a decimal constant value added or subtracted. Finally, the present value of the location counter designated by \$ may appear as the first operand with an optional decimal constant added or subtracted. The second operand is an addressing mode identifier (IXB field) or a null field. If the IXB field is null, the comma after the first operand field may be omitted.

The allowable mode identifiers are as follows:

IDENTIFIER	VALUE	NAME
BR	1	Base register relative
X	2	Indexed, program counter relative
XBR	3	Indexed, base register relative
I	4	Indirect, program counter relative
IBR	5	Indirect, base register relative
IX	6	Indirect, post indexing, program counter relative
M	7	Immediate addressing

The first operand may optionally be preceded by * which indicates indirect addressing.

Examples:

```

A      LDA      0,M
A      LDX      -17,M
A      LDE      >8D,M
A      @LDA     ->8000,M
A      @LDM     >FF0,M
A 603  LDA      ITEM
A      MPY      3,M
A      LDM      IPART+17,X
A      LDA      REAL,BR
A      LDE      REAL+1,BR
A      BIX      &605
A      BRU      &45-2,X
A      BRU      $-6
A      LDA      $+25,IX

```

When NAME is referenced within a FORTRAN function or subroutine subprogram, and NAME is one of the dummy parameters appearing in the subroutine or function definition statement, the operand acquired is the *address* of the actual parameter as it appears in the subprogram reference, and not the *value* of the parameter itself.

Example:

```

        INTEGER FUNCTION RAND (J,L)
A       LDA     J
A       STA     &5
A       LDA     L
A       STA     &10
A       LDA     *&10
A       AND     *&5
A       STA     RAND
        RETURN
A       5      DATA  0
A       10     DATA  0
        END
```

NOTE

The only valid method to return from a subprogram is by use of the RETURN statement.

In the case of function subprograms, the compiler generates code to load the proper registers with the function value prior to generating the calling sequence for the return. Hence, the assembly language programmer must *store* all function values before writing the RETURN statement. They may not be left in registers.

4-3 REGISTER-TO-REGISTER INSTRUCTIONS.

The following instructions of the register-to-register type are allowed:

```

RSU  RIN  RCL
RAD  RCA  RAN
RCO  ROR  RDE
RIV  RMO  REX
REO
```

As operands for the instructions, the following symbols may be used to identify source and destination registers.

A - Accumulator register
E - Extension register
X - Index register
M - Mask register
S - Storage register
L - Link register
B - Base register
P - Program register

4-4 INDICATOR SKIP INSTRUCTIONS.

The following are allowed:

SEQ	SGE	SNV
SNE	SLE	SOC
SGT	SOV	SNC
SLT		

No operands are allowed for these instructions.

4-5 REGISTER SKIP INSTRUCTIONS.

All of the register skip types are allowed.

SZE	SMI	SOO
SNZ	SEV	SNO
SPL	SOD	

The allowable operand is one of the register symbols listed under 4-3.

4-6 SHIFT INSTRUCTIONS.

The following are allowed:

ARA	LLD	CRX
ARD	RTO	CRM
LRA	RTZ	CRS
LRD	LTO	CRL
ALA	LTZ	CRB
ALD	CRA	CLD
LLA	CRE	CRD

The allowable operand is a number less than 32, indicating the shift count.

4.7 PSEUDO-OP INSTRUCTIONS.

The following IAL pseudo-operations are allowed:

BRS
BRR
REF
DATA

The pseudo-op BRS has as its operand a label, e.g.,

```
A    BRS &450
```

The pseudo-op BRR must have a null operand field, e.g.,

```
A    BRR
```

The DATA pseudo-op may have a variety of operand types. The following are allowed:

- a. A character string enclosed in single quotes.
- b. Hexadecimal integer constants from 0 to FFFF, inclusive, identified by preceding back slash (\) or greater-than (>) symbol.
- c. Decimal integer constants from 0 to 32,767, inclusive.
- d. Decimal real numbers including E-format representation.
- e. Symbolic variable names, whether in (labeled or blank) common storage or noncommon storage, and external references. An integer constant may be added or subtracted.
- f. Numeric labels constructed in the customary FORTRAN format, preceded by an ampersand. An integer constant may be added or subtracted.
- g. The current location counter value specified by dollar sign (\$), plus or minus an integer constant.

Examples:

```
A    DATA  'THIS IS A MESSAGE.'  
A    DATA  6,3.72,3.14159,8.E3  
A    DATA  &3878  
A    DATA  ICOUNT,>3F7F,ALPHA
```


NOTE

Executable instructions which are not included in the IAL set may be coded as operands in the DATA statements. These DATA statements must be properly placed in a series of executable codes for sequential execution.

The REF pseudo-op may have as its operand a symbolic field of six characters or less representing an external subroutine, function name, or defined entry point of an assembly language program.

```
A      REF    $F4RRP,EXT1,INOUT
```

4-8 RESTRICTIONS.

The programmer is allowed to change the contents of any register. However, since the code generated by the compiler uses base-register-relative and indexed addressing to reference scalars, array elements, and labels, some warnings are in order. Indiscriminate use of registers may cause incorrect variables to be referenced, with no error indication being given and no hope of recovery.

WARNING

If the programmer plans to use the base register in an IAL program, he must save the contents of this register in a local data area and restore the saved contents as illustrated in the following sequence.

```

      .
      .
      .      FORTRAN CODING
A      BRS      &705      BEGIN IAL CODING
A      RMO      B,A
A      STA      &706
A      LDA      &707
C
C      SET LOCAL BASE PAGE
A      RMO      A,B
A 705  . . . . .
      .
      .
C      RESTORE REGISTERS
A      LDA      &706
A      RMO      A,B
A      BRU      &708
A 706  DATA    0
A 707  DATA    &705
C 708
C      CONTINUE FORTRAN CODING
```

APPENDIX A

TABLE A-1. FORTRAN BASIC EXTERNAL FUNCTIONS

Function	Definition	No. Arguments	Name	Arg. Type	Function Type
Exponential	e^a	1	EXP	Real	Real
			DEXP	Double	Double
			CEXP	Complex	Complex
Natural Log	$\log_e(a)$	1	ALOG	Real	Real
			DLOG	Double	Double
			CLOG	Complex	Complex
Common Log	$\log_{10}(a)$	1	ALOG10	Real	Real
			DLOG10	Double	Double
Sine	$\sin(a)$	1	SIN	Real	Real
			DSIN	Double	Double
			CSIN	Complex	Complex
Cosine	$\cos(a)$	1	COS	Real	Real
			DCOS	Double	Double
			CCOS	Complex	Complex
Hyperbolic Tan	$\tanh(a)$	1	TANH	Real	Real
Square Root	$(a)^{1/2}$	1	SQRT	Real	Real
			DSQRT	Double	Double
			CSQRT	Complex	Complex
Arctangent	$\arctan(a)$	1	ATAN	Real	Real
		1	DATAN	Double	Double
	$\arctan(a_1/a_2)$	2	ATAN2	Real	Real
		2	DATAN2	Double	Double
Remaindering*	$a_1 \text{ (mod } a_2)$	2	DMOD	Double	Double
Modulus	$ c $	1	CABS	Complex	Real

*The function DMOD (a_1, a_2) is defined as $a_1 - [a_1/a_2]a_2$ when $[a_1/a_2]$ is the integer whose magnitude does not exceed (a_1/a_2) .

TABLE A-2. FORTRAN INTRINSIC FUNCTIONS

Function	Definition	No. Args.	Name	Arg. Type	Function Type
Absolute Value	$ a $	1	ABS	Real	Real
			IABS	Integer	Integer
			DABS	Double	Double
Truncation	Sign of a times largest integer $\leq a $	1	AINT	Real	Real
			INT	Real	Integer
			DINT	Double	Integer
Remaindering*	$a_1 \text{ (mod } a_2 \text{)}$	2	AMOD MOD	Real Integer	Real Integer
Choose Largest Value	$\text{MAX}(a_1, a_2, \dots, a_n)$	≥ 2	AMAX0	Integer	Real
			AMAX1	Real	Real
			MAX0	Integer	Integer
			MAX1	Real	Integer
			DMAX1	Double	Double
Choose Smallest Value	$\text{MIN}(a_1, a_2, \dots, a_n)$	≥ 2	AMIN0	Integer	Real
			AMIN1	Real	Real
			MIN0	Integer	Integer
			MIN1	Real	Integer
			DMIN1	Double	Double
Float	Convert Integer to Real	1	FLOAT	Integer	Real
Fix	Convert Real to Integer	1	IFIX	Real	Integer
Transfer of Sign	Sign a_2 times $ a_1 $	2	SIGN	Real	Real
			ISIGN	Integer	Integer
			DSIGN	Double	Double
Positive Difference	$a_1 - \text{MIN}(a_1, a_2)$	2	DIM	Real	Real
			IDIM	Integer	Integer
Double to Real		1	SNGL	Double	Real
Real Part of Complex		1	REAL	Complex	Real
Imaginary Part of Complex		1	AIMAG	Complex	Real
Real to Double		1	DBLE	Real	Double
Two Reals to Complex	$a_1 + a_2 \sqrt{-1}$	2	CMPLX	Real	Complex
Complex Conjugate	$a - ib$	1	CONJG	Complex	Complex

*See footnote at bottom of Table A-1.

TABLE A-3. FORTRAN FORMAT EDITOR

Entry	Function
\$F4EFI	End File
\$F4RWD	Rewind
\$F4SIO	Stop Input/Output
\$F4RBN	Read - Binary
\$F4RBD	Read - ASCII
\$F4WBN	Write - Binary
\$F4WBD	Write - ASCII
\$F4IIO	Integer/Logical Input/Output
\$F4RIO	Real Input/Output
\$F4CIO	Complex Input/Output
\$F4DIO	Double-Precision Input/Output
\$F4IUA	Integer/Logical Unsubscripted Array Input/Output
\$F4RUA	Real Unsubscripted Array Input/Output
\$F4CUA	Complex Unsubscripted Array Input/Output
\$F4DUA	Double-Precision Unsubscripted Array Input/Output
\$F4BSP	Backspace

NOTE

Stop and pause library functions
are also available.

\$F4STP Stop

\$F4PAU Pause

APPENDIX B

RUN-TIME ERROR MESSAGES

FORTRAN FORMAT EDITOR ERROR MESSAGES

MESSAGE	MEANING
ILLEGAL FORMAT CHARACTER	Illegal character encountered in run-time format statement.
ILLEGAL INPUT CHARACTER	Illegal character encountered in input stream during READ execution.
FORMAT PARENTHESIS ERROR	Run-time format statement contains unbalanced set of parenthesis.
UNDER/OVER FLOW	Real number in input stream is outside range of numbers allowed.
** WARNING ** RECORD SIZE ERROR	User program attempting to input or output record containing greater than 132 characters.
INPUT (OUTPUT, REWIND, BACKSP, ENDFILE) UNIT LIMIT ERROR	I/O was attempted on a FORTRAN unit greater than 7.
OUT OF DATA	End-of-file encountered during a READ operation.
ERR. OR WRTBIN (WRTBCD, REDBCD, REDBIN, ENDFIL, REWIND, BACKSP) COMM IGND ON UNIT B0 (1, 2, . . . , 7) JOB ABORTED	Hardware error, or attempted operation is illegal on device specified, i.e., rewind card punched is illegal.

FLOATING-POINT ARITHMETIC PACKAGE ERROR MESSAGES

MESSAGE	MEANING
UNDERFLOW	The result of a floating-point operation is smaller than system allows, i.e., less than 1.E-39.
OVERFLOW	The result of a floating-point operation is greater than system allows, i.e., greater than 1.E+39.
DIVIDE CHECK	An attempt was made to perform a floating-point division by zero.

NOTE

The user program is terminated in all of the above cases except the ** WARNING ** condition.

COMMENT SHEET

MODEL 980A COMPUTER
FORTRAN
LIBRARY AND
IN-LINE ASSEMBLY LANGUAGE

FROM:

NAME _____ OFFICE/DEPT NO. _____

CITY/STATE _____ DATE _____

To make this manual more useful to you, we want your comments: what additional information should be included in the manual; what description or figure could be clarified; what subject requires more explanation; what presentation is particularly helpful to you; and so forth. Thank you.

CUT ALONG LINE

NO POSTAGE NECESSARY IF MAILED IN U.S.A.

FOLD ON TWO LINES (LOCATED ON REVERSE SIDE), STAPLE AND MAIL

FOLD

ATTENTION: TECHNICAL PUBLICATIONS
M/S 670

TEXAS INSTRUMENTS INCORPORATED
DIGITAL SYSTEMS DIVISION
P.O. BOX 1444 HOUSTON, TEXAS 77001

Postage will be paid by

BUSINESS REPLY MAIL
No postage necessary if mailed in the United States

FIRST CLASS
Permit No. 6189
Houston, Texas

FOLD

Sales and Service Offices of Texas Instruments are located throughout the United States and in major countries overseas. Contact the Digital Systems Division, Texas Instruments Incorporated, P.O. Box 1444, Houston, Texas 77001, or call (713) 494-5115, for the location of the office nearest to you.



Texas Instruments reserves the right to make changes at any time to improve design and supply the best product possible.

TEXAS INSTRUMENTS
INCORPORATED

TI-508-1.8M-3/73